Graded Lab #1 (VERSION.2)

AI (E11)

Artificial Intelligence – Lab | University of Central Punjab

Time: 1 hour	Total Marks: 25	Language: Python
Name: _____	Reg. No: _____	Date: _____

Plagiarism Policy

- **This is an individual assessment. All code must be written by you during the lab session.**
- **Copying code from another student, sharing your code, or allowing others to copy it is plagiarism and is treated as cheating by both parties.**
- **Using AI tools, online solutions, websites, messaging apps, or unauthorised notes to generate or obtain answers is not allowed.**
- **Code that is identical or suspiciously similar (including identical variable names, comments or mistakes) will be treated as plagiarism.**
- **You may be asked to explain your code. If you cannot explain it, you will receive zero marks for that question.**
- **Any student found guilty will receive zero marks in this graded lab, and the case will be reported according to university rules.**

Question 1: Digit Sum (10 marks)

Task

Write a function **extract_digits_sum(text)** that returns the **sum** of all the digit characters found in a string. All other characters (letters, spaces, symbols) are ignored.

Rules

- **Do not use isdigit(), isnumeric() or sum().**
- Each digit is added on its own: in "a22" the digits are 2 and 2, so the answer is 4, not 22.
- **The function must return the number, not just print it.**
- If the string has no digits, return 0.
- **It must run in O(n).** Use a single loop and look at each character only once.

Hint

- Use for ch in text: to visit each character one by one.
- A digit character such as '7' is a string, so convert it with int() before adding it.
- Do not use sum as a variable name, because it replaces a built-in Python function.

Samples

Sample 1

```
Input:  "a12b"
Output: 3
```

Sample 2

```
Input:  "a1b22c3"
Output: 8
```

(The digits are 1, 2, 2 and 3, so 1 + 2 + 2 + 3 = 8.)

Input	Expected output
"a12b"	3
"a1b22c3"	8
"abc"	0
"2024"	8
""	0

Question 2: Car Fuel Tracker (15 marks)

Task

Ask the user for the fuel tank size (litres) and the fuel currently in the car (litres). Store them in variables **tank_size** and **available_fuel_in_tank**. Then show a menu in a **while** loop until the user chooses Exit:

```
1. Refuel
2. Drive
3. Show fuel
4. Trips possible
5. Exit
```

Terms

- **tank_size**: the maximum litres the tank can hold.
- **available_fuel_in_tank**: the litres of fuel currently in the car.
- **refuling_amount**: the litres entered by the user for one refuel action.
- **driving_amount**: the litres entered by the user for one drive action.
- **per_trip**: the litres one trip uses (used in option 4).

Option	What it does	Marks
`1. Refuel`	`Uses refuel(available_fuel_in_tank, refuling_amount, tank_size)`	`1.25`
`2. Drive`	`Uses drive(available_fuel_in_tank, driving_amount)`	`1.25`
`3. Show fuel`	`Prints the current available_fuel_in_tank`	`1.25`
`4. Trips possible`	`Uses trips_possible(available_fuel_in_tank, per_trip)`	`10`
`5. Exit`	`Prints a goodbye message and leaves the loop`	`1.25`

Functions

Do not use classes or global variables.

- **refuel(available_fuel_in_tank, refuling_amount, tank_size)** returns the new fuel after adding refuling_amount. If refuling_amount is zero or negative, or the new fuel would be more than tank_size, it returns available_fuel_in_tank unchanged.
- **drive(available_fuel_in_tank, driving_amount)** returns the new fuel after subtracting driving_amount. If driving_amount is zero, negative, or greater than available_fuel_in_tank, it returns available_fuel_in_tank unchanged.
- **trips_possible(available_fuel_in_tank, per_trip)** uses a loop to count how many full trips the fuel can cover, where each trip uses per_trip litres. It returns the count. If per_trip is zero or negative, it returns the message Enter Valid amount!!

Call	Expected output
`refuel(30, 10, 50)`	`40`
`refuel(45, 10, 50)`	`45`
`refuel(30, -5, 50)`	`30`
`drive(40, 15)`	`25`
`drive(40, 60)`	`40`
`drive(40, 0)`	`40`
`trips_possible(50, 12)`	`4`
`trips_possible(100, 30)`	`3`
`trips_possible(10, 12)`	`0`
`trips_possible(50, 0)`	`Enter Valid amount!!`

Rules

- The available_fuel_in_tank variable must always hold the current fuel. After every refuel or drive, store the returned value back into it (for example, available_fuel_in_tank = refuel(available_fuel_in_tank, refuling_amount, tank_size)), so the next menu choice uses the updated amount.
- Option 3 must show the current available_fuel_in_tank, and option 4 must use it.
- Option 4 only reads the fuel. It must not change available_fuel_in_tank.
- Your program must defend against errors and crashes caused by invalid input.
- If refuelling or driving fails, print a clear message.

Hint

- input() always returns a string, so convert it with int() or float().
- Use break to leave the loop when the user chooses Exit.

Sample run

```
Enter tank size: 50
Enter current fuel: 30
Choose option: 1
Enter refuelling amount: 10
Fuel: 40
Choose option: 1
Enter refuelling amount: 30
Refuelling failed: invalid amount or tank would overflow
Choose option: 2
Enter driving amount: 15
Fuel: 25
Choose option: abc
Invalid input. Please enter a number.
Choose option: 3
Fuel: 25
Choose option: 4
Enter fuel used per trip: 8
Trips possible: 3
Choose option: 3
Fuel: 25
Choose option: 5
Goodbye!
```